

SLASH_COMMANDS_USER_GUIDE

Slash Commands User Guide

HelixCode Command System

Version: 1.0 Last Updated: November 7, 2025 Feature Status:  Production Ready

Table of Contents

- 1. [Introduction](#)
- 2. [Quick Start](#)
- 3. [Built-in Commands](#)
- 4. [Command Reference](#)
- 5. [Advanced Usage](#)
- 6. [Creating Custom Commands](#)
- 7. [Examples](#)
- 8. [Troubleshooting](#)
- 9. [FAQ](#)

Introduction

Slash commands in HelixCode provide a powerful way to trigger specific actions and workflows without natural language ambiguity. Similar to commands in Slack or Discord, slash commands start with / and offer structured, predictable behavior.

Benefits

- **Speed:** Fast, predictable actions
- **Clarity:** No ambiguity in what you want
- **Autocomplete:** Tab completion for commands and flags
- **Consistency:** Same syntax across all commands
- **Composability:** Combine with @ mentions for power

Available Commands

Command	Aliases	Purpose
/newtask	/nt, /task	Create a new task with preserved context
/condense	/smol, /compact, /summarize	Compress chat history to save tokens
/newrule	/rule, /guideline	Generate project rules from conversation

Command	Aliases	Purpose
/reportbug	/bug, /issue	File a bug report with system info
/workflows	/wf, /flow	List or execute workflows
/deepplanning	/deepplan, /dp, /architect	Enter extended planning mode

Quick Start

Basic Command

/help

Shows all available commands.

Command with Arguments

/newtask Implement user authentication

Creates a new task with the description “Implement user authentication”.

Command with Flags

/newtask Fix database connection --priority high

Creates a high-priority task.

Command with Both

/condense --keep-last 10 --preserve-code

Condenses chat history keeping last 10 messages and preserving code blocks.

Built-in Commands

/newtask - Create New Task

Purpose: Create a new development task while preserving relevant context from the current conversation.

Syntax:

```
/newtask <description>
/newtask <description> --priority <level>
/newtask <description> --link-previous
/newtask <description> --transfer-files
```

Arguments: - description: Task description (required)

Flags: `--priority`: Task priority (low, normal, high, critical) - default: normal `--link-previous`: Link to current task as a continuation `--transfer-files`: Transfer file references from chat history to new task

Examples:

Basic task:

```
/newtask Refactor authentication module
```

High-priority task:

```
/newtask Fix memory leak in worker pool --priority critical
```

Linked task:

```
/newtask Add tests for authentication --link-previous --priority high
```

With file transfer:

```
/newtask Document the API endpoints --transfer-files
```

Output:

```
Created new task: Fix memory leak in worker pool
Priority: critical
Task ID: task-12345
```

Use Cases: - Breaking down large features into smaller tasks - Creating follow-up tasks during code review - Tracking bugs discovered during development - Planning implementation steps

Integration: - Tasks appear in task dashboard - Can be exported to GitHub Issues, Jira, etc. - Tracked across sessions

/condense - Compress Chat History

Purpose: Summarize and compress conversation history to save tokens while preserving important details.

Aliases: `/smol`, `/compact`, `/summarize`

Syntax:

```
/condense
/condense --keep-last <N>
/condense --preserve-code
/condense --preserve-errors
/condense --ratio <0.1-1.0>
```

Flags: `--keep-last N`: Keep last N messages uncompressed (default: 5) `--preserve-code`: Keep all code blocks intact (default: false) `--preserve-errors`: Keep all error messages intact (default: false) `--ratio`: Target compression ratio (default: 0.5 = 50% reduction)

Examples:

Basic compression:

```
/condense
```

Keep last 10 messages:

```
/condense --keep-last 10
```

Preserve important content:

```
/condense --preserve-code --preserve-errors
```

Aggressive compression:

```
/condense --ratio 0.3 --keep-last 3
```

Output:

Condensing 45 messages (keeping last 5 uncompressed)

Before: ~12,500 tokens

After: ~6,200 tokens

Compression: 50.4%

What Gets Condensed: - Repetitive back-and-forth - Verbose explanations - Redundant context
- Intermediate iterations

What's Preserved: - Recent messages (configurable) - Code blocks (if flag set) - Error messages
(if flag set) - Key decisions and outcomes - File references

Use Cases: - Long conversations approaching token limit - Before starting a new major topic -
When switching contexts - Performance optimization

Best Practices: - Condense before starting unrelated work - Keep more messages for active
debugging - Always preserve code when debugging - Review condensed summary before
continuing

/newrule - Generate Project Rules

Purpose: Analyze conversation patterns and generate coding rules or guidelines based on
corrections and preferences.

Aliases: /rule, /guideline

Syntax:

```
/newrule [category]  
/newrule [category] --global  
/newrule [category] --name <custom-name>
```

Arguments: - category: Rule category (optional) - default: general

Categories: - coding-style: Code formatting and style preferences - testing: Testing
requirements and patterns - architecture: Architectural decisions - documentation:
Documentation standards - general: General development practices

Flags: - -global: Save as global rule (applies to all projects) - default: workspace - - -name: Custom rule name - default: <category>-rules

Examples:

From current conversation:

```
/newrule coding-style
```

Global rule:

```
/newrule testing --global
```

Custom name:

```
/newrule "error handling" --name robust-errors
```

How It Works:

The command analyzes your chat history for patterns: 1. **Corrections:** When you say “instead”, “should”, “prefer”, “always”, “never” 2. **Preferences:** When you specify “use X” or “don’t use Y” 3. **Repeated Issues:** Things mentioned multiple times

Output:

Generating coding-style rule: coding-style-rules
Location: .helixrules/coding-style-rules.md

Analyzed: 47 messages

Found: 8 patterns

- "Always use tabs instead of spaces"
- "Prefer const over var for immutable variables"
- "Never use var, prefer const or let"
- ...

Generated Rule File:

```
# Coding Style Rules
```

Generated from conversation on 2025-11-07

```
## Indentation
```

- Always use tabs instead of spaces
- Tab width: 4 spaces

```
## Variable Declarations
```

- Never use ``var``
- Prefer ``const`` for immutable values
- Use ``let`` only when reassignment needed

```
## Error Handling
```

- Always check errors explicitly

- Use named error variables

...

Use Cases: - Capturing team conventions - Documenting decisions made during code review - Creating project-specific guidelines - Building coding standards from practice

Integration: - Rules stored in `.helixrules/` directory - Automatically loaded in future sessions - Can be committed to version control - Shared across team

/reportbug - File Bug Report

Purpose: Create a comprehensive bug report with system information, logs, and reproduction steps.

Aliases: `/bug`, `/issue`

Syntax:

```
/reportbug <description>
/reportbug <description> --title <title>
/reportbug <description> --labels <label1,label2>
/reportbug <description> --attach-logs
/reportbug <description> --auto-submit
```

Arguments: - `description`: Bug description (optional) - default: "Bug report from HelixCode"

Flags: - `--title`: Custom issue title - default: "Bug: " - `--labels`: Comma-separated labels - default: "bug" - `--attach-logs`: Include recent logs - default: true - `--auto-submit`: Automatically submit to GitHub - default: false (opens for review)

Examples:

Basic bug report:

```
/reportbug "LLM timeout error"
```

With custom title and labels:

```
/reportbug --title "Memory leak in worker pool" --labels bug,critical
```

Auto-submit:

```
/reportbug "Crash on startup" --auto-submit
```

Collected Information: - **System:** OS, architecture, CPU count, Go version, HelixCode version - **Logs:** Last 50 log entries (configurable) - **Reproduction Steps:** Extracted from recent chat history - **Context:** Current session, project, working directory

Generated Report:

Description

LLM timeout error when processing large files

System Information

go_version: go1.26.3 os: darwin arch: arm64 num_cpu: 8 helix_version: 0.1.0 timestamp: 2025-11-07T14:32:15Z

Reproduction Steps

1. Load large file (>10,000 lines)
2. Request refactoring
3. Timeout occurs after 60s

Recent Logs

[2025-11-07 14:31:45] INFO: Processing file: large_file.go [2025-11-07 14:31:47] DEBUG: LLM request sent (12,450 tokens) [2025-11-07 14:32:45] ERROR: Timeout waiting for response

Expected Behavior

Should handle large files with streaming or chunking

Actual Behavior

Times out after 60 seconds

Generated by HelixCode /reportbug command

Use Cases: - Quick bug reporting during development - Capturing error context automatically - Sharing issues with team - Creating GitHub issues

Integration: - GitHub API integration (if configured) - GitLab support (coming soon) - Jira integration (coming soon) - Local markdown file export

/workflows - Workflow Management

Purpose: List, execute, or manage development workflows.

Aliases: /wf, /flow

Syntax:

```
/workflows                                # List all workflows
/workflows <name>                         # Execute workflow
/workflows --list                          # List with details
/workflows <name> --params <params>
/workflows <name> --async
/workflows --status <workflow-id>
/workflows --cancel <workflow-id>
```

Arguments: - name: Workflow name (optional for list, required for execute)

Flags: - - - list: Show detailed workflow information - - - params: Pass parameters (JSON or key=value) - example: unit , integration or type=unit - - - async: Run workflow in

background - --status: Check status of running workflow - --cancel: Cancel a running workflow

Built-in Workflows:

Workflow	Description	Steps
planning	Analyze requirements and create specifications	3
building	Generate code and manage dependencies	4
testing	Run unit, integration, and E2E tests	5
refactoring	Analyze and optimize code structure	3
debugging	Identify and fix issues	4
deployment	Build, package, and deploy to targets	6

Examples:

List workflows:

```
/workflows
/workflows --list
```

Execute workflow:

```
/workflows planning
```

Execute with parameters:

```
/workflows testing --params "unit,integration"
```

Background execution:

```
/workflows deployment --async
```

Check status:

```
/workflows --status workflow-12345
```

Cancel workflow:

```
/workflows --cancel workflow-12345
```

Output (List):

Found 6 available workflows:

- 1. planning - Analyze requirements and create technical specifications (3
- 2. building - Generate code and manage dependencies (4 steps)
- 3. testing - Run unit, integration, and end-to-end tests (5 steps)
- 4. refactoring - Analyze and optimize code structure (3 steps)
- 5. debugging - Identify and fix issues (4 steps)
- 6. deployment - Build, package, and deploy to targets (6 steps)

Use /workflows <name> to execute

Output (Execute):

Executing planning workflow...

Step 1/3: Requirements Analysis ✓

Step 2/3: Architecture Design ⌚

...

Use Cases: - Structured development processes - Automated testing pipelines - Code quality checks - Deployment automation - Refactoring large codebases

Custom Workflows: - Create workflows in `.helix/workflows/` - YAML or JSON format - Full documentation: [Custom Workflows Guide](#)

/deepplanning - Extended Planning Mode

Purpose: Enter comprehensive planning mode with detailed analysis, architecture design, and implementation planning.

Aliases: `/deepplan`, `/dp`, `/architect`

Syntax:

```
/deepplanning <topic>
/deepplanning <topic> --depth <1-5>
/deepplanning <topic> --output <file>
/deepplanning <topic> --include-diagrams
/deepplanning <topic> --focus <areas>
/deepplanning <topic> --constraints <constraints>
/deepplanning --resume <plan-id>
```

Arguments: - `topic`: Planning topic (required unless `--resume`)

Flags: - `--depth`: Planning depth 1-5 (default: 3) - higher = more detailed - `--output`: Save plan to file (markdown or JSON) - `--include-diagrams`: Generate architecture diagrams (ASCII/Mermaid) - `--focus`: Focus areas (architecture,security,performance,scalability) - `--constraints`: Constraints (budget=low,timeline=2weeks) - `--resume`: Resume previous planning session

Planning Phases:

1. **Requirements Analysis:** Gather and analyze requirements
2. **Architecture Design:** Design system architecture and components
3. **Technology Selection:** Choose appropriate technologies
4. **Implementation Planning:** Break down into tasks and milestones
5. **Risk Assessment:** Identify risks and mitigation strategies
6. **Resource Estimation:** Estimate time, team size, resources

Examples:

Basic planning:

```
/deepplanning "new authentication system"
```

Detailed planning:

```
/deepplanning "microservices architecture" --depth 5
```

With output file:

```
/deepplanning "database migration" --output plan.md
```

With diagrams:

```
/deepplanning "event-driven system" --include-diagrams
```

Focused planning:

```
/deepplanning "API redesign" --focus architecture,security,performance
```

With constraints:

```
/deepplanning "mobile app" --constraints "budget=low,timeline=3months"
```

Resume planning:

```
/deepplanning --resume plan-12345
```

Output:

Starting deep planning for: new authentication system

Planning depth: 3

Focus areas: architecture, implementation

Phase 1/6: Requirements Analysis ✓

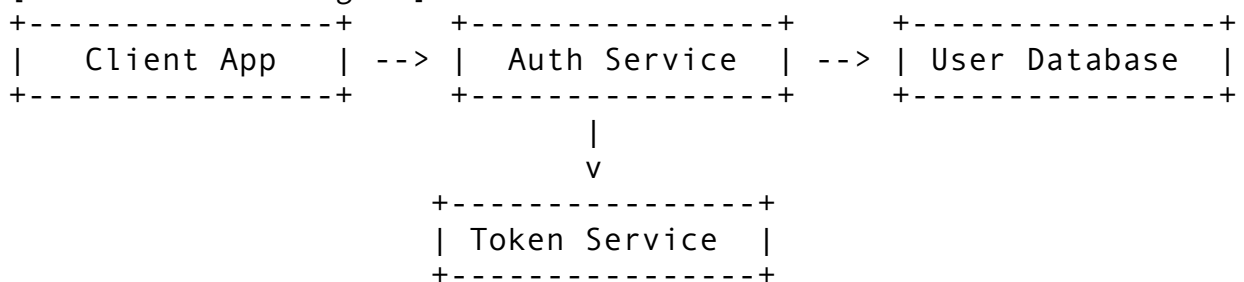
- User authentication (email/password)
- OAuth 2.0 support (Google, GitHub)
- JWT token-based sessions
- Role-based access control
- ...

Phase 2/6: Architecture Design ⌚

Components:

- Auth Service (handles authentication)
- Token Service (JWT generation/validation)
- User Service (user management)
- Role Service (RBAC)

[Architecture Diagram]



...

Saved Plan (if –output specified):

Authentication System - Deep Planning

****Created:**** 2025-11-07

****Depth:**** 3

****Status:**** Complete

Requirements Analysis

Functional Requirements

1. User authentication with email/password
2. OAuth 2.0 integration (Google, GitHub)
3. JWT token-based sessions
4. Role-based access control (RBAC)

...

Architecture Design

System Components

...

Technology Selection

...

Implementation Plan

Phase 1: Core Authentication (2 weeks)

- [] Setup database schema
- [] Implement user registration
- [] Implement login/logout

...

Risk Assessment

...

Resource Estimation

...

Use Cases: - Large feature planning - System architecture design - Refactoring planning - Technology evaluation - Team onboarding (documentation)

Integration: - Plans saved to `.helix/plans/` - Can be exported to Notion, Confluence - Generate tasks from plans - Track implementation progress

Command Reference

Command Syntax

```
/command [arguments] [--flag value] [--boolean-flag]
```

Components:

1. **Command Name:** Starts with / (required)
2. **Arguments:** Positional values (space-separated)
3. **Flags:** Named options with -- prefix

Flag Formats:

--flag value	# Flag with value
--flag=value	# Alternative syntax
--boolean-flag	# Boolean flag (value=true)

Quoting:

Use quotes for arguments with spaces:

```
/newtask "Fix bug in authentication handler"  
/deepplanning "Design microservices architecture"
```

Autocomplete:

Press Tab to autocomplete: - Command names - Flag names - File paths (for some commands)

Advanced Usage

Combining Commands with @ Mentions

```
/newtask Fix the issue in @file[internal/auth/handler.go] --priority high
```

Chaining Commands

Execute multiple commands in sequence:

```
/condense --ratio 0.3  
# (wait for completion)  
/newtask Implement the suggested improvements
```

Workflow Pipelines

```
/workflows testing --params unit  
# (after tests pass)  
/workflows deployment --async
```

Planning to Implementation

```
/deepplanning "user dashboard" --output dashboard-plan.md
# (review plan)
/newtask Implement user dashboard --link-previous
```

Iterative Refinement

```
/newrule coding-style
# (review generated rules)
# (continue conversation with corrections)
/newrule coding-style # Regenerate with new patterns
```

Creating Custom Commands

Command Structure

Custom commands are defined in `.helix/commands/`:

```
# .helix/commands/deploy.yaml
name: deploy
description: "Deploy to production"
aliases: ["ship", "release"]

steps:
  - type: workflow
    name: testing
  - type: shell
    command: "make build"
  - type: shell
    command: "kubectl apply -f k8s/"
  - type: notify
    message: "Deployment complete!"
```

More Information

See [Custom Commands Guide](#) for full documentation.

Examples

Example 1: Feature Development

```
# Start with planning
/deepplanning "user profile feature" --depth 3 --output profile-plan.md

# Create tasks from plan
/newtask Implement user profile API --priority high
/newtask Implement user profile UI --link-previous
```

```
/newtask Add profile tests --link-previous
```

```
# During development, add rules  
/newrule "API design"
```

```
# When history gets long  
/condense --keep-last 10 --preserve-code
```

Example 2: Bug Investigation

```
# File bug report  
/reportbug "Memory leak in background worker" --labels bug,critical
```

```
# Run debugging workflow  
/workflows debugging
```

```
# Create fix task  
/newtask Fix memory leak in background worker --priority critical
```

Example 3: Code Review

```
# Condense old conversation  
/condense --ratio 0.5
```

```
# Start review workflow  
/workflows refactoring
```

```
# Generate style guide from feedback  
/newrule coding-style --name team-style-guide --global
```

Troubleshooting

Command Not Found

Problem:

Error: command 'unknowncmd' not found

Solutions: 1. Check spelling: /help shows all commands 2. Use autocomplete: type / and press Tab 3. Check aliases: some commands have multiple names

Invalid Flag

Problem:

Error: unknown flag: --invalid

Solutions: 1. Check command help: /command --help 2. Use correct syntax: --flag value or --flag=value 3. Boolean flags don't need values: --preserve-code not --preserve-code true

Missing Required Argument

Problem:

Error: description is required

Solutions: 1. Provide required arguments 2. Use quotes for multi-word arguments 3. Check command syntax in this guide

Workflow Not Found

Problem:

Unknown workflow: invalid-workflow

Solutions: 1. List workflows: `/workflows --list` 2. Check spelling 3. Custom workflows: verify `.helix/workflows/` directory

FAQ

Q: Can I create my own slash commands? A: Yes! See [Custom Commands Guide](#).

Q: Do slash commands work with all LLM providers? A: Yes, commands are processed before sending to the LLM.

Q: Can I use @ mentions in slash commands? A: Yes! Example: `/newtask Fix @file[handler.go]`

Q: How do I see all available commands? A: Type `/help` or press `/` followed by Tab.

Q: Are slash commands case-sensitive? A: No, `/NewTask`, `/newtask`, and `/NEWTASK` all work.

Q: Can slash commands be abbreviated? A: Not built-in commands, but you can use aliases (e.g., `/nt` for `/newtask`).

Q: Do slash commands save history? A: Yes, all commands are logged and can be reviewed.

Q: Can I undo a slash command? A: Some commands support undo (e.g., `/condense`). Check command documentation.

Q: How do I cancel a running workflow? A: Use `/workflows --cancel <workflow-id>`

Q: Can slash commands be scripted? A: Yes, via CLI mode. See [CLI Scripting Guide](#).

See Also

- [@ Mentions User Guide](#)
- [Custom Commands Guide](#)
- [Workflow Development Guide](#)

- [HelixCode Configuration](#)
 - [API Documentation](#)
-

Document Version: 1.0 **Last Updated:** November 7, 2025 **Feedback:** [GitHub Issues](#)

Sources verified

Sources verified 2026-05-29: <https://go.dev/dl/> (go1.26.3 latest stable Go; 1.24 past support) ;
project go.mod (root go 1.25.2, inner go 1.26) + CLAUDE.md §3.1 (PostgreSQL 15+).